

```
elif weight_diff > 0:
    print(f"▶ 목표까지 {weight_diff:.1f}kg 남았습니다. 조금만 더
    힘내세요!")
else:
    print("▶ 이미 목표 체중에 달성하셨습니다. 대단해요!")

print(f"▶ 오늘 먹은 음식들: {daily_meals}")
print("*** * 50) 이게 내가 지금까지 작성한 코딩 내용인데 여기에
파이썬에서 while문이란 if문을 조합해서, 사용자가 '종료'를 선택
하기 전까지 반복해서 메뉴판을 보여주고 기능을 실행하는 제어
구조를 만들고 싶어. 전체적인 코드 틀을 어떻게 잡아야 할까?
```

질문하신 대로 단발성으로 끝나는 V2.0 코드를 지속 가능한 프로그램으로 업그레이드하려면, `while True` 무한 루프와 `if-elif-else` 조건문을 조합한 '메뉴 시스템 제어 구조'가 필요합니다.

사용자가 특정 번호(예: 5번 종료)를 누르기 전까지 프로그램이 계속 반복해서 실행되도록 만드는 전체적인 코드 틀(뼈대)과 설계 원리를 안내해 드릴게요.

1. 무한 루프 메뉴 시스템의 핵심 구조

프로그램이 계속 실행되려면 아래와 같은 흐름으로 코드가 짜여야 합니다.

- 1 `while True:` 를 선언하여 무한 루프를 시작합니다.
- 2 루프 내부에서 사용자에게 메뉴판을 출력하고, 어떤 기능을 쓸지 번호를 입력받습니다.
- 3 `if-elif-else` 를 통해 사용자가 입력한 번호에 맞는 코드가 실행되도록 분기합니다.
- 4 사용자가 종료 번호를 누르면 `break` 를 실행하여 루프를 탈출하고 프로그램을 끝냅니다.

2. 무한 루프 메뉴 시스템 표준 코드 틀 (템플릿)

작성하신 V2.0 코드를 이 틀에 나누어 담으시면 됩니다. 전체적인 구조는 다음과 같습니다.

Python

```
while True:
    # 1. 메뉴판 출력
    print("\n" + "=" * 50)
    print("      Smart Fit & Diet Coach V3.0 ")
    print("=" * 50)
    print(" 1. 사용자 신체 정보 등록 및 수정")
    print(" 2. BMI 지수 및 건강 등급 분석")
    print(" 3. 오늘의 식단 기록 추가 (누적)")
    print(" 4. 전체 건강 관리 기록 조회")
    print(" 5. 프로그램 종료")
    print("=" * 50)
```

```

while True:
    # 1. 메뉴판 출력
    print("\n" + "=" * 50)
    print("    Smart Fit & Diet Coach V3.0 ")
    print("=" * 50)
    print(" 1. 사용자 신체 정보 등록 및 수정")
    print(" 2. BMI 지수 및 건강 등급 분석")
    print(" 3. 오늘의 식단 기록 추가 (누적)")
    print(" 4. 전체 건강 관리 기록 조회")
    print(" 5. 프로그램 종료")
    print("=" * 50)

    # 2. 사용자 선택 입력받기
    choice = input("원하는 기능의 번호를 선택하세요: ")

    # 3. 조건문을 통한 기능 분기
    if choice == "1":
        print("\n[안내] 1번 신체 정보 등록 기능을 실행합니다.")
        # 이 자리에 V2.0의 정보 입력 및 리스트 저장 로직 삽입

    elif choice == "2":
        print("\n[안내] 2번 BMI 분석 기능을 실행합니다.")
        # 이 자리에 V2.0의 BMI 계산 및 등급 판정 로직 삽입

    elif choice == "3":
        print("\n[안내] 3번 식단 기록 추가 기능을 실행합니다.")
        # 이 자리에 V2.0의 식단 입력 로직 삽입

    elif choice == "4":
        print("\n[안내] 4번 전체 건강 관리 기록 조회 기능을 실행합니다.")
        # 이 자리에 V2.0의 최종 분석 보고서 및 특별 메시지 출력 로직 삽입

    elif choice == "5":
        # 4. 루프 탈출 조건 (break)
        print("\n프로그램을 종료합니다. 오늘도 건강한 하루 되세요! 💪")
        break # 이 break를 만나야 while True 무한 루프가 끝납니다.

    else:
        # 5. 예외 처리 (1~5 외의 다른 값을 입력했을 때)
        print("\n[오류] 잘못된 번호입니다. 1~5 사이의 숫자를 입력해 주세요.")

```

💡 캡처 및 과제 작성을 위한 팁

- 제어 흐름의 변화: V2.0에서는 위에서 아래로 순차 실행되고 프로그램이 바로 꺼졌지만, 이 구조를 도입 하면 사용자가 5번을 누르기 전까지 1번 -> 3번 -> 4번 -> 2번 등 원하는 순서대로 자유롭게 기능을 반복 실행할 수 있게 됩니다.
- 주의할 점: 이 구조를 그대로 적용할 때, 사용자가 1번을 입력하기 전에 2번이나 4번을 누르면 변수가 없어서 에러가 나거나, 3번을 누를 때마다 기존 데이터가 초기화되는 현상이 발생할 수 있습니다. 이 부분은 데이터의 선언 위치와 예외 처리 코드를 통해 보완해야 합니다.

while True 무한 루프를 써서 메뉴판을 만들었는데, 3번 메뉴에서 식단을 입력할 때마다 기존에 저장했던 식단 리스트 데이터가 계속 초기화돼서 사라져. 입력한 데이터가 프로그램이 꺼질 때까지 계속 누적되게 하려면 리스트 변수를 어디에 선언해야 해?

매우 중요한 문제를 발견하셨습니다! 이 현상은 파이썬을 처음 배울 때 무한 루프 환경에서 가장 많이 겪는 대표적인 오류 중 하나로, 변수의 선언 위치(Lifecycle/Scope) 때문에 발생합니다.

결론부터 말씀드리면, 입력한 데이터가 리셋되지 않고 프로그램이 꺼질 때까지 누적되게 하려면 리스트 변수를 while True: 루프의 '외부(상단)'에 선언해야 합니다.

왜 이런 일이 발생하는지 원인과 해결 방법을 코드로 명확히 짚어드릴게요.

❌ 데이터가 계속 초기화되었던 원인 (잘못된 위치)

기존 생각대로 while 문 내부나 3번 메뉴 조건문 안에 리스트를 선언하면 다음과 같은 상황이 벌어집니다.

Python

```
while True:
    # (메뉴판 출력 생략)
    choice = input("선택: ")

    if choice == "3":
        # ❌ 예러 원인: 3번을 누를 때마다 빈 리스트가 새로 만들어지며 기존 데이터가 덮어쓰워짐(초기화)
        daily_meals = []
        daily_meals.append(input("오늘 메뉴 추가: "))
        print(f"현재 식단: {daily_meals}")
```

- 이유: 사용자가 3번 메뉴를 선택할 때마다 `daily_meals = []` 코드가 매번 새로 실행됩니다. 즉, 기존에 채워져 있던 리스트는 버려지고 항상 빈 리스트로 리셋된 후 새 음식을 넣기 때문에 데이터가 한 개 이상 쌓이지 않는 것입니다.

데이터 누적을 위한 올바른 해결 방법 (while문 외부 선언)

프로그램이 시작될 때 리스트를 딱 한 번만 만들어두고, while 루프 안에서는 `append()` 를 통해 데이터만 추가해야 합니다.

데이터 누적을 위한 올바른 해결 방법 (while문 외부 선언)

프로그램이 시작될 때 리스트를 딱 한 번만 만들어두고, while 루프 안에서는 `append()` 를 통해 데이터만 추가해야 합니다.

Python

```
# 정답: 데이터 보존을 위해 while 루프 외부(최상단)에 변수 선언!
# 프로그램이 실행되는 동안 이 리스트는 단 한 번만 생성됩니다.
daily_meals = []
user_data = []

while True:
    print("\n=== 메뉴 ===")
    print("3. 오늘의 식단 기록 추가")
    print("4. 전체 건강 관리 기록 조회")
    print("5. 프로그램 종료")

    choice = input("원하는 기능 선택: ")

    if choice == "3":
        print("\n[식단 기록 추가]")
        meal_input = input("기록할 음식 이름을 입력하세요: ")

        # while문 외부의 리스트에 기존 데이터를 유지하며 차곡차곡 누적(append)
        daily_meals.append(meal_input)
        print(f"▶ '{meal_input}'이(가) 추가되었습니다.")

    elif choice == "4":
        # 조회해 보면 누적된 식단 리스트가 그대로 출력됩니다.
        print(f"\n▶ 현재까지 누적된 식단: {daily_meals}")

    elif choice == "5":
        print("프로그램 종료")
        break
```

💡 핵심 요약

- while 문 내부에 변수를 선언하면 → 바퀴(루프)가 돌 때마다 변수가 새로 리셋됩니다.
- while 문 외부에 변수를 선언하면 → 루프가 아무리 돌아도 변수가 초기화되지 않고 데이터가 안전하게 누적됩니다.

이 개념을 적용하시면 3번 메뉴로 식단을 계속 추가하고, 4번 메뉴로 통합 조회했을 때 먹은 음식들이 리스트에 예쁘게 쌓여있는 모습을 확인하실 수 있습니다!

프로그램을 실행하고 사용자가 1번을 안 누른 상태에서 2번이나 4번을 먼저 누르면 리스트가 비어 있어서 에러가 나면서 프로그램이 꺼져버려. 이걸 막으려면 어떻게 코드를 짜야 안전할까?

프로그램을 무한 루프 구조로 바꾸면서 발생하는 가장 대표적인 예외 상황을 짚어내셨네요!

1번 메뉴를 건너뛰고 2번이나 4번을 누르면, `user_data` 리스트가 비어 있거나(`[]`) 내부에 아무 데이터도 없기 때문에 리스트의 특정 인덱스를 참조하려 할 때 `IndexError: list index out of range` 가 발생하며 프로그램이 강제 종료됩니다.

이를 막기 위해 프로그램의 비정상 종료를 원천 차단하는 방어적 프로그래밍(Defensive Programming) 기법을 적용해야 합니다. 가장 깔끔하고 직관적인 방법은 '등록 상태를 기억하는 스위치(플래그 변수)'를 사용하는 것입니다.

💡 해결 방법: `is_registered` 플래그 변수 활용

프로그램 시작 시 정보가 등록되지 않았다는 의미로 `is_registered = False` 를 만들어두고, 1번 메뉴에서 입력을 완수하면 `True` 로 바꿔주는 스위치 역할을 부여하는 것입니다.

이후 2번과 4번 메뉴에서는 이 스위치가 `True` 인지 먼저 검사하고, 만약 `False` 라면 경고 메시지와 함께 `continue` 를 실행해 에러가 나기 전에 다시 메뉴로 돌려보냅니다.

🔗 실제 코드 적용 예시

Python

```
# [1] 프로그램 시작 전 플래그(스위치) 변수와 데이터 리스트 선언
user_data = []
daily_meals = []
is_registered = False # 처음에는 등록되지 않은 상태(False)

while True:
    print("\n=== Smart Fit & Diet Coach ===")
    print("1. 사용자 신체 정보 등록")
    print("2. BMI 지수 및 건강 등급 분석")
    print("4. 전체 건강 관리 기록 조회")
    print("5. 프로그램 종료")

    choice = input("번호를 선택하세요: ")

    if choice == "1":
        name = input("이름: ")
        age = int(input("나이: "))
        height = float(input("키(cm): "))
        weight = float(input("현재 몸무게(kg): "))
```



```

elif choice == "1":
    name = input("이름: ")
    age = int(input("나이: "))
    height = float(input("키(cm): "))
    weight = float(input("현재 몸무게(kg): "))
    target_weight = float(input("목표 몸무게(kg): "))

    user_data = [name, age, height, weight, target_weight]

    # 1번 입력을 정상적으로 마쳤으므로 스위치를 켜(True)
    is_registered = True
    print(f"\n[성공] {name}님의 신체 정보가 등록되었습니다.")

elif choice == "2":
    # ✖ 에러 방어: 등록 스위치가 꺼져 있다면(False) 진입 차단!
    if not is_registered:
        print("\n[알림] 먼저 1번 메뉴에서 신체 정보를 등록해 주세요.")
        continue # 아래의 계산 코드를 실행하지 않고, 즉시 while문 시작점으로 점프!

    # 1번이 실행되어 데이터가 안전하게 있을 때만 이 코드가 실행됨
    height_m = user_data[2] / 100
    bmi = user_data[3] / (height_m ** 2)
    print(f"\n▶ {user_data[0]}님의 BMI 지수: {bmi:.2f}")

elif choice == "4":
    # ✖ 에러 방어: 통합 조회 시에도 신체 정보가 없으면 차단
    if not is_registered:
        print("\n[알림] 등록된 신체 정보가 없습니다. 1번 메뉴를 먼저 이용해 주세요.")
        print(f"\n▶ 현재까지 기록된 식단: {daily_meals}")
        continue

    print(f"\n▶ {user_data[0]}님의 통합 보고서...")
    print(f"\n▶ 현재까지 누적된 식단: {daily_meals}")

elif choice == "5":
    print("프로그램을 종료합니다.")
    break

```

핵심 키워드 정리 (캡처/과제용 핵심 이론)

- 1 플래그(Flag) 변수 (`is_registered`): * 프로그램의 특정 상태(정보 등록 여부)를 이진값 (`True` / `False`)으로 기록하여 제어 흐름을 조절하는 변수입니다.
- 2 `continue` 키워드의 역할:
 - 루프 안에서 `continue` 를 만나면 그 아래에 있는 복잡한 계산이나 리스트 참조 코드 (`user_data[2]`)를 전부 무시하고, 즉시 `while True` 조건문의 처음(메뉴 출력)으로 강제 리턴시킵니다. 덕분에 비어 있는 리스트를 건드려서 발생하는 `IndexError` 를 완벽하게 방어할 수 있습니다.

이렇게 구성하시면 사용자가 실수로 2번이나 4번을 먼저 눌러도 프로그램이 튕기지 않고 친절하게 안내 문구를 띄운 뒤 정상 작동하게 됩니다!